

Build Task — Retriever Module — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python — how a real codebase would actually write it).

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three small files that work together: one that splits text into chunks, one that builds a searchable store out of those chunks, and one that searches it.

`chunking.py` 's job: turn one long piece of text into a list of smaller pieces, using a method you can swap out — don't hardcode one splitting method inline, write it as its own named function so you can compare methods later.

`ingest.py` 's job: read every document in a folder, chunk each one, embed every chunk, and store the embeddings — while remembering, for each chunk, which file and which position in that file it came from.

`retriever.py` 's job: one function, `retrieve(query, k)`, that embeds a question and hands back the `k` closest chunks. If there's nothing stored yet, it should hand back an empty list, not blow up.

Things to look up and use: - `pathlib.Path(folder).glob("*")` — to find every file in your `docs/` folder. - Plain string slicing (`text[i:i+300]`) for a fixed-size chunker. - `text.split("\n\n")` for a paragraph chunker, then strip each piece and throw away empty ones. - The OpenAI embeddings API, model `text-embedding-3-small` — turns a piece of text into a list of numbers. - `chromadb.Client()` and `collection.add(...)` / `collection.query(...)` — a simple local vector store, no server needed. - LangChain's `Chroma` and `OpenAIEmbeddings` — a wrapper around the same idea, with less code to write yourself.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You're building three small, separate pieces that get used together: a **chunker**, an **ingest step**, and a **retriever**. Keep them as three separate files, each doing one job.

`chunking.py` needs at least two different splitting methods, each its own named function. `ingest.py` 's job is: read every document in a folder, run each one through a chunking function, embed every chunk (same embedding model you'll use for questions later), and store the resulting vectors. The part that's easy to forget: track, for every single chunk, which source file it came from and which position (index) it held within that file's chunk list. `retriever.py` 's job is: one function, `retrieve(query, k)`, that embeds the incoming question with the *same* embedding model used during ingest, searches the store, and returns the top `k` matches — returning `[]` immediately if the store has zero chunks.

Here's what to actually go look at: - `pathlib.Path(folder).glob("*.txt")` (and again for `"*.md"`) — the standard way to list files of a given type in a folder. - **String slicing** (`text[i:i+chunk_size]`) is all `chunk_by_chars()` needs. - `text.split("\n\n")` is the core of `chunk_by_paragraph()`. - `openai.embeddings.create(model="text-embedding-3-small", input=text)` — accepts a *list* of strings too, so you can embed a whole batch of chunks in one call. - `chromadb.Client()`, then `collection.add(ids=[...], embeddings=[...], documents=[...], metadatas=[...])` — `metadatas` is where `source_file` and `chunk_index` belong. `collection.query(query_embeddings=[...], n_results=k)` searches it. - **LangChain's** `Chroma.from_texts(texts, embedding, metadatas=[...])` and `OpenAIEmbeddings()` — a higher-level wrapper. `.similarity_search_with_score(query, k=k)` returns LangChain `Document` objects, exactly the type Project 3's agent tools expect back.

Try sketching, in plain English, what happens to one document on its way into the store, and what happens to one question on its way to becoming a result list.

Think past "ingest once and it works," too. Ask: **what happens if `build_vector_store()` runs a second time on the same folder — does the store end up with every chunk duplicated?**

With `chromadb`, calling `collection.add()` again with the same `ids` either raises or silently overwrites, depending on version and setup — neither is "add a second identical copy," but neither is automatically safe either unless you design for it. The real design answer: build each chunk's `id` deterministically from something stable (`f"{file_name}-{chunk_index}"`), as the hints already do) rather than a random UUID, so re-running ingest on an unchanged file naturally lands on the *same* IDs instead of creating duplicates — and decide explicitly whether a re-run should skip existing IDs, or delete-then-re-add to pick up an edited file.

A second real question worth sketching: what happens when `doc_folder` has thousands of files? Calling the embeddings API once per file (rather than batching many chunks per call) is slow and wastes the API's ability to embed a whole list at once. Collect chunks across *all* files first, then make one (or a few, capped at the API's batch-size limit) embedding call for the whole batch — not one call per file or per chunk.

Difference between Basic and Intermediate: Basic gets one clean ingest run working correctly. Intermediate also asks what happens the *second* time you run it (idempotency, via deterministic IDs) and what happens at real scale (batching embedding calls instead of one-per-file) — two questions that don't show up with a 3-file test folder, but absolutely show up in a real knowledge base that grows and changes over time.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
chunking.py:
    function chunk_by_chars(text, chunk_size=300):
        step through text in windows of chunk_size characters
        return the list of pieces

    function chunk_by_paragraph(text):
        split text on blank lines
        strip each piece, drop empty ones
        return the list of pieces

ingest.py:
    function build_vector_store(doc_folder, chunk_fn=chunk_by_paragraph):
        make a new empty collection/store
        for each .txt or .md file in doc_folder:
            read its text
            chunks = chunk_fn(text)
            for each chunk, remember its index in this file's chunk list
            embed all the chunks
            add them to the store, along with source_file and chunk_index for each
        return the store

retriever.py:
    function retrieve(query, k=3):
        if the store is empty: return []
        embed the query
        search the store for the k closest chunks
        build a small Document for each result: text, source_file, chunk_index, score
        return the list of Documents
```

The trickiest part — tracking which file and which chunk-index each piece of text came from, so it survives all the way into the search results:

```
def build_vector_store(doc_folder, chunk_fn):
    all_texts = []
    all_metadatas = []
```

```

for file_name in os.listdir(doc_folder):
    if not (file_name.endswith(".txt") or file_name.endswith(".md")):
        continue
    path = os.path.join(doc_folder, file_name)
    with open(path, "r") as f:
        text = f.read()
    chunks = chunk_fn(text)
    for chunk_index, chunk_text in enumerate(chunks):
        all_texts.append(chunk_text)
        all_metadatas.append({
            "source_file": file_name, "chunk_index": chunk_index,
        })

# all_texts[i] and all_metadatas[i] always describe the same chunk –
# keep them in step with each other
return all_texts, all_metadatas

```

And the empty-store guard for `retrieve()` :

```

def retrieve(query, k=3):
    if collection.count() == 0:
        return []
    # ... otherwise search normally

```

Try finishing the rest yourself before looking at the Solution.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`chunking.py` :

```
function chunk_by_chars(text, chunk_size=300):
    result = []
    step i from 0 to len(text) in steps of chunk_size:
        append text[i:i+chunk_size] to result
    return result

function chunk_by_paragraph(text):
    pieces = text split on "\n\n"
    stripped = [strip each piece]
    return [piece for piece in stripped if piece is not empty]
```

`ingest.py` :

```
function build_vector_store(doc_folder, chunk_fn=chunk_by_paragraph):
    create a new, empty vector store / collection
    find every .txt and .md file under doc_folder
    for each file:
        read its full text
        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            keep track of: chunk_text, source_file=this file's name, chunk_index
    embed all collected chunk texts in one batch call
    add every (embedding, chunk_text, source_file, chunk_index) into the store
    return the store
```

`retriever.py` :

```
function retrieve(query, k=3):
    if the store has zero chunks stored:
        return []
    embed the query with the same embedding model used in ingest.py
    search the store for the k closest chunk embeddings
    for each result, build a Document with: text, source_file, chunk_index, score
    return the list of Documents, ordered closest-first
```

One small piece to get you unstuck — keeping each chunk's text lined up with its own `source_file` and `chunk_index` all the way through embedding and storage:

```
from pathlib import Path

def build_vector_store(doc_folder: str, chunk_fn) -> tuple[list[str], list[dict]]
    all_texts: list[str] = []
    all_metadatas: list[dict] = []

    for path in Path(doc_folder).glob("*"):
        if path.suffix not in (".txt", ".md"):
            continue
        text = path.read_text()
        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })

    return all_texts, all_metadatas
```

The empty-store check has to happen *before* you try to search — searching an empty store is exactly the case the requirements call out as "must not crash."

Try finishing the rest yourself before looking at the full Solution below.

Once that's working, sketch the deterministic-ID scheme — this is what makes re-running ingest safe:

```
chunk_id = f"{path.name}-{chunk_index}" # stable across re-runs on the same file
```

Then sketch the batched-embedding call: collect `all_texts` across every file first (as the hints already do), and make exactly one `client.embeddings.create(model=..., input=all_texts)` call for the whole batch, instead of one call inside the per-file loop. Write both pieces yourself before checking the Solution.

Difference between Basic and Intermediate: Basic builds a working ingest-and-search pipeline for a clean, one-time run. Intermediate adds the two things that only matter once this runs more than once, or on more than a handful of files — a deterministic ID scheme so re-ingesting doesn't duplicate, and one batched embedding call instead of one call per file.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Read both depths below — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — raw `chromadb`

Story — `chunking.py` / `ingest.py` / `retriever.py` : this is Project 3's Retriever module — every practice exercise above (chunking, embedding similarity, the vector-store search) folds into these three files. **If not:** Project 3 would be the first place any of these pieces ever had to work together, with no smaller version to trust.

```
# chunking.py

def chunk_by_chars(text, chunk_size=300):
    chunks = []
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i = i + chunk_size
    return chunks

def chunk_by_paragraph(text):
    raw_pieces = text.split("\n\n")
    chunks = []
    for piece in raw_pieces:
        stripped = piece.strip()
        if stripped != "":
            chunks.append(stripped)
    return chunks
```

```
# ingest.py
import os
import chromadb
from openai import OpenAI
from chunking import chunk_by_paragraph
```

```

client = OpenAI()
chroma_client = chromadb.Client()

def build_vector_store(doc_folder, chunk_fn=chunk_by_paragraph):
    collection = chroma_client.create_collection(name="docs")

    all_texts = []
    all_metadatas = []
    all_ids = []

    for file_name in os.listdir(doc_folder):
        if not (file_name.endswith(".txt") or file_name.endswith(".md")):
            continue
        path = os.path.join(doc_folder, file_name)
        with open(path, "r") as f:
            text = f.read()

        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": file_name, "chunk_index": chunk_index,
            })
            all_ids.append(file_name + "-" + str(chunk_index))

    if len(all_texts) == 0:
        return collection

    response = client.embeddings.create(
        model="text-embedding-3-small", input=all_texts,
    )
    embeddings = [item.embedding for item in response.data]

    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    return collection

```

```

# retriever.py
from openai import OpenAI

client = OpenAI()

```

```
def retrieve(collection, query, k=3):
    if collection.count() == 0:
        return []

    response = client.embeddings.create(
        model="text-embedding-3-small", input=[query],
    )
    query_embedding = response.data[0].embedding

    results = collection.query(query_embeddings=[query_embedding], n_results=k)

    documents = []
    texts = results["documents"][0]
    metadatas = results["metadatas"][0]
    distances = results["distances"][0]

    for i in range(len(texts)):
        documents.append({
            "text": texts[i],
            "source_file": metadatas[i]["source_file"],
            "chunk_index": metadatas[i]["chunk_index"],
            "score": distances[i],
        })

    return documents
```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — raw `chromadb`, with type hints

Story — `ingest.py` / `retriever.py` (Intermediate): Basic proved the pipeline works; this version makes it safe to build on — typed signatures, and `all_ids` built from `f"{path.name}-{chunk_index}"` instead of left implicit. **If not:** a typo in a dict key would fail silently instead of being caught, and nothing about the ID scheme would hint that it needs to be deterministic (Approach 2 below explains why).

```
# chunking.py

def chunk_by_chars(text: str, chunk_size: int = 300) -> list[str]:
    chunks: list[str] = []
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += chunk_size
    return chunks

def chunk_by_paragraph(text: str) -> list[str]:
    raw_pieces = text.split("\n\n")
    chunks = [piece.strip() for piece in raw_pieces]
    return [piece for piece in chunks if piece != ""]
```

```
# ingest.py
from pathlib import Path
from typing import Callable

import chromadb
from openai import OpenAI

from chunking import chunk_by_paragraph

EMBEDDING_MODEL = "text-embedding-3-small"

client = OpenAI()
chroma_client = chromadb.Client()
```

```

def build_vector_store(
    doc_folder: str,
    chunk_fn: Callable[[str], list[str]] = chunk_by_paragraph,
):
    collection = chroma_client.create_collection(name="docs")

    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []

    for path in Path(doc_folder).glob("*"):
        if path.suffix not in (".txt", ".md"):
            continue
        text = path.read_text()

        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })
            # why: deterministic, not a random UUID -- a re-run on an
            # unchanged file lands on the same ID instead of duplicating
            all_ids.append(f"{path.name}-{chunk_index}")

    if not all_texts:
        return collection

    response = client.embeddings.create(model=EMBEDDING_MODEL, input=all_texts)
    embeddings = [item.embedding for item in response.data]

    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    return collection

```

```

# retriever.py
from openai import OpenAI

EMBEDDING_MODEL = "text-embedding-3-small"

client = OpenAI()

def retrieve(collection, query: str, k: int = 3) -> list[dict]:

```

```

if collection.count() == 0:
    return [] # when: an empty store must not crash, per the requirements

response = client.embeddings.create(model=EMBEDDING_MODEL, input=[query])
query_embedding = response.data[0].embedding

results = collection.query(query_embeddings=[query_embedding], n_results=k)

texts = results["documents"][0]
metadatas = results["metadatas"][0]
distances = results["distances"][0]

documents: list[dict] = []
for text, metadata, distance in zip(texts, metadatas, distances):
    documents.append({
        "text": text,
        "source_file": metadata["source_file"],
        "chunk_index": metadata["chunk_index"],
        "score": distance,
    })

return documents

```

Difference from Basic: same logic, with full type hints, and `all_ids` built as `f"{path.name}-{chunk_index}"` — a deterministic ID, not just a detail, since it's the piece that makes re-ingesting the same folder safe (see Approach 2 below).

Approach 2 — idempotent re-ingest: delete-then-add on a re-run

Story: calling `collection.add()` again with the same IDs either raises or silently overwrites, depending on version and setup — neither is "add a second identical copy," but neither is automatically safe either unless you design for it. **If not:** re-running ingest after editing one document would leave the store in a state nobody could predict without checking the `chromadb` version installed.

```

def build_vector_store(
    doc_folder: str,
    chunk_fn=chunk_by_paragraph,
    collection_name: str = "docs",
):
    # why: delete-then-recreate starts every run from a clean, known state --
    # simpler and safer than trying to diff old vs. new chunks by hand
    try:
        chroma_client.delete_collection(name=collection_name)
    except Exception:
        pass # how: didn't exist yet -- fine, this is the first run

```

```

collection = chroma_client.create_collection(name=collection_name)

all_texts: list[str] = []
all_metadatas: list[dict] = []
all_ids: list[str] = []

for path in Path(doc_folder).glob("*"):
    if path.suffix not in (".txt", ".md"):
        continue
    text = path.read_text()
    chunks = chunk_fn(text)
    for chunk_index, chunk_text in enumerate(chunks):
        all_texts.append(chunk_text)
        all_metadatas.append({
            "source_file": path.name, "chunk_index": chunk_index,
        })
        all_ids.append(f"{path.name}-{chunk_index}")

if not all_texts:
    return collection

# why: one batched call for every chunk across every file, not one
# call per file -- see Approach 3's Story for what this costs otherwise
response = client.embeddings.create(model=EMBEDDING_MODEL, input=all_texts)
embeddings = [item.embedding for item in response.data]

collection.add(
    ids=all_ids, embeddings=embeddings,
    documents=all_texts, metadatas=all_metadatas,
)
return collection

```

Re-running `build_vector_store()` on an edited folder now produces a clean, correct store every time — no duplicate chunks from re-running, and no stale chunks left over from a file that was deleted since the last ingest.

Approach 3 — LangChain's `Chroma` wrapper, same idempotency idea

Story: Project 3's agent tooling expects LangChain `Document` objects back from retrieval — building the store with LangChain's own `Chroma` wrapper here avoids a conversion step later, while keeping the exact same delete-then-rebuild safety as Approach 2. **If not:** Project 3 would need a translation layer between this module's raw dicts and whatever shape the agent tools actually expect.

```

def build_vector_store(
    doc_folder: str,

```

```

    chunk_fn=chunk_by_paragraph,
    persist_directory: str = "./chroma_db",
):
    embeddings = OpenAIEmbeddings(model=EMBEDDING_MODEL)

    # why: a fresh Chroma instance at the same persist_directory, cleared
    # first, gives the same "clean re-run" guarantee as Approach 2's
    # delete-then-create
    store = Chroma(
        embedding_function=embeddings, persist_directory=persist_directory,
    )
    store.delete_collection()
    store = Chroma(
        embedding_function=embeddings, persist_directory=persist_directory,
    )

    all_texts: list[str] = []
    all_metadatas: list[dict] = []

    for path in Path(doc_folder).glob("*"):
        if path.suffix not in (".txt", ".md"):
            continue
        text = path.read_text()
        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })

    if not all_texts:
        return store

    return Chroma.from_texts(
        texts=all_texts, embedding=embeddings,
        metadatas=all_metadatas, persist_directory=persist_directory,
    )

```

Difference from Approach 1, and between Approaches 2/3: Approach 1 builds a store that works correctly the *first* time. Approaches 2 and 3 make re-running ingest safe (delete-then-rebuild, rather than silently appending duplicates) and keep the embedding call batched across the whole folder — the two things that actually matter once this runs more than once against a folder of documents that changes over time, which is the normal case for a real knowledge base. Approach 3 additionally returns LangChain `Document` objects directly, matching what Project 3's agent tools expect.

Which one should you actually use?

For Project 3, use Approach 3 with LangChain's `Chroma` wrapper — the agent tooling in later documents expects `Document` objects, and using the same type here avoids a conversion step. Keep Approach 2/3's delete-then-rebuild pattern regardless of which wrapper you use — a `build_vector_store()` that silently duplicates on a second run is a real bug waiting to surface the first time someone re-runs ingest after editing a document, which will happen.